D.P.M.S.
Artificial Intelligence

Εργασία 1

Course: Natural Language Processing - NLP

Ioannis Petrousov

March, 2026

A. Tokens, Types, Zipf's Law

1

	NLTK	spaCy	Bert
#tokens	93356	97152	117968
#types	11017	10445	8544
TTR	0.1180	0.1075	0.0724
Hapax legomena	5615	5072	2915
Hapax dislegomena	1702	1659	1381

The method which creates the largest vocabulary is the one with the highest #types count, in this case NLTK. However, it's important to note that while this number is the highest of all other methods, it's not necessary the most accurate. Specifically, word_tokenize() uses the Punkt sentence tokenization model to splits the text into individual elements. This model uses a word-based tokenization approach which also splits punctuation marks, including ",", "." and "'" into separate tokens thus increasing the #types count. The same can be observed for the tokenization method spaCy uses. This results into creating tokens of the form:

```
In [1]: "." in tokens
Out[1]: True

In [2]: "," in tokens
Out[2]: True

In [3]: "'" in tokens
Out[3]: True

In [4]: "it's" in tokens
Out[4]: False

In [5]: "'s" in tokens
Out[5]: True
```

On the other hand, Bert uses a more modern subword tokenization approach - WordPiece. The method effectively recognizes and constructs larger words (from smaller) thus optimizing #types count.

TTR expresses vocabulary diversity. The higher values observed in NLTK and spaCy indicate a richer vocabulary. However, BERT has a vocabulary consisting of longer words which might have an increased meaning compared to the ones from the aforementioned methods.

Both spaCy and NLTK follow Zipf's law: $5615/11017 = 50\%$ and $5072/10445 = 48\%$ so half of the vocabulary appears only once in the tokens. BERT shows a lower percentage of Hapax Legomena (34%), likely because its subword units are designed to be highly reusable across different words, reducing the number of unique, single-use types compared to word-based tokenizers.

2

I selected the first sentence: ”Pierre Vinken, 61 years old, will join the board as a nonexecutive director Nov. 29.”. Following are the results from the tokenization.

```
NLTK tokens:
['Pierre', 'Vinken', ',', '61', 'years', 'old', ',', 'will',
'join', 'the', 'board', 'as', 'a', 'nonexecutive', 'director',
'Nov.', '29', '.']
===============
spaCy tokens:
['Pierre', 'Vinken', ',', '61', 'years', 'old', ',', 'will',
'join', 'the', 'board', 'as', 'a', 'nonexecutive', 'director',
'Nov.', '29', '.']
===============
BERT tokens:
['Pierre', 'Vin', '##ken', ',', '61', 'years', 'old', ',', 'will',
'join', 'the', 'board', 'as', 'a', 'none', '##xe', '##cut', '##ive',
'director', 'Nov', '.', '29', '.']
===============
```

We can see that both NLTK and spaCy managed to split almost all the words correctly because they use a word-based tokenization approach. However, BERT uses a statistical method to construct tokens. In the example we can see that it broke the word ”Vinken” into 2: ”Vin”, ”##ken”. This happening because the word does not exist in its vocabulary. Also ”nonexecutive” was broken into 4 subwords: ”none”, ”##xe”, ”##cut”, and ”##ive” because it greedy-matched separate words from it’s vocabulary starting with the word ”none”. It couldn’t match the rest of the word (”xecutive”) so it broke it into 3 pieces using ”##” in front of each piece.

3

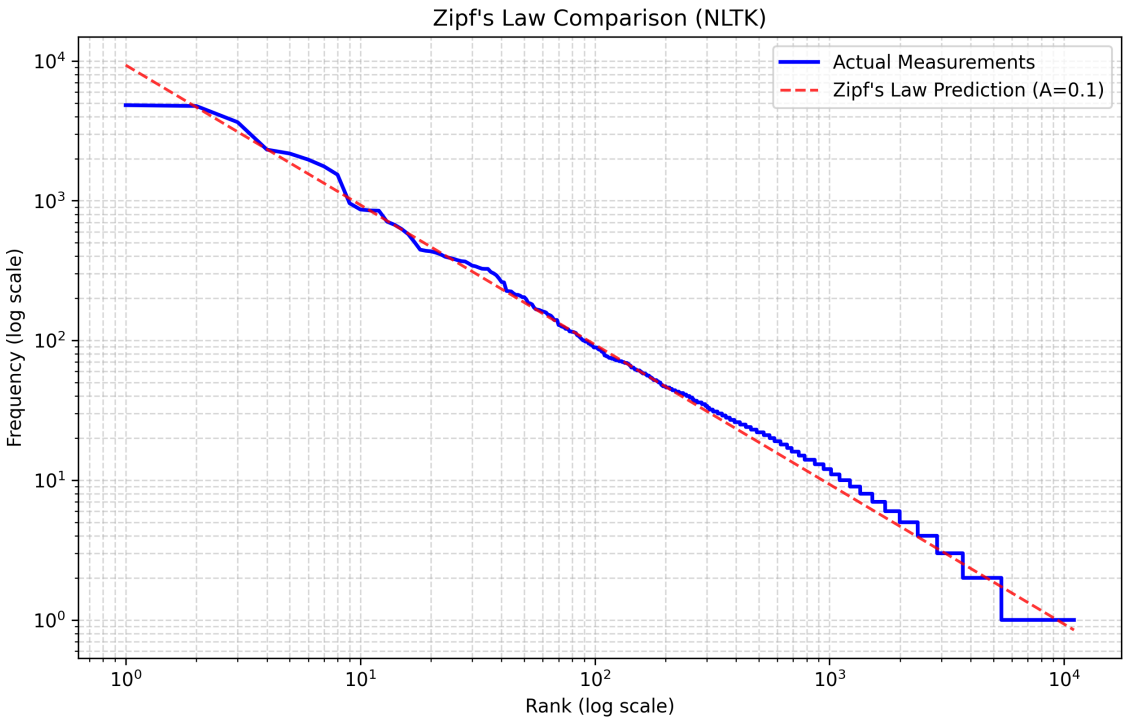
	NLTK Type	NLTK Freq	spaCy Type	spaCy Freq	BERT Type	BERT Freq
0	,	4823	.	5020	.	6363
1	the	4757	,	4823	,	5026
2	.	3645	the	4763	the	4767
3	of	2318	of	2319	’	4117
4	to	2175	to	2180	of	2320
5	a	1967	a	1991	to	2226
6	in	1760	in	1773	a	2150
7	and	1534	and	1541	in	1940
8	”	959	”	1372	-	1733
9	’s	865	-	1231	and	1549
10	for	851	’s	865	s	1356
11	that	848	for	851	for	870
12	$	708	NaN	<NA>	that	848
13	is	672	NaN	<NA>	NaN	<NA>

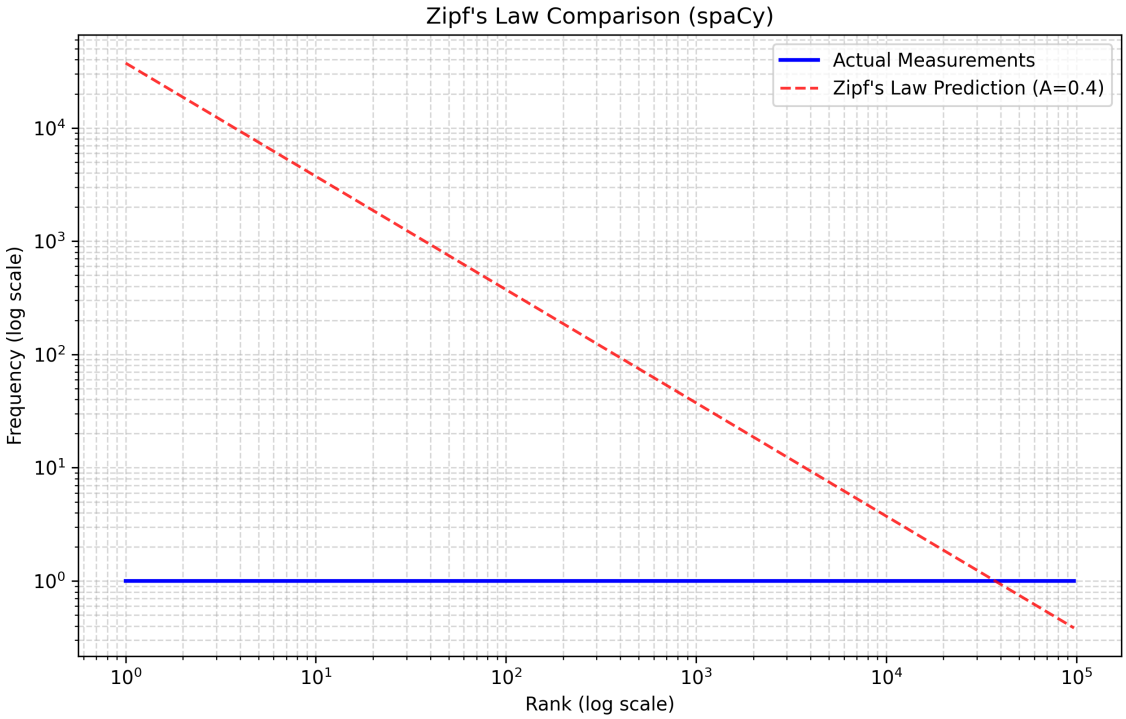
I extracted the number of common tokens programmatically and it was found to be 9. See Python notebook for details.

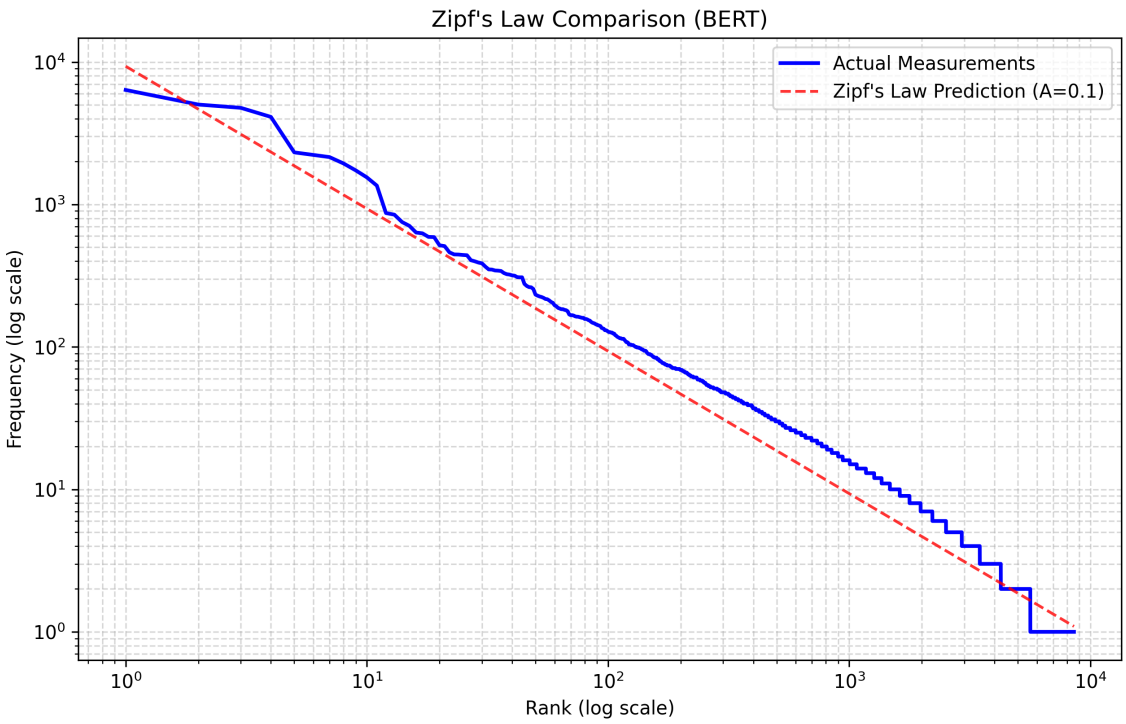
4

Following are the optimal calculated values. I used Mean Square Error - MSE to measure the distance between the actual and calculated values for the token appearance frequency.

	NLTK	**spaCy**	**BERT**
A	0.1	0.4	0.1







5

Following is the prompt I used for ChatGPT:

> "I have the tokens from 3 libraries: NLTK, spaCy and BERT.

Generate Python code for the following assignment: Σύμφωνα με τον Νόμο του Zipf, το γινόμενο της πιθανότητας ενός type επί την θέση του στη λίστα των types (ταξινομημένη κατά φθίνουσα σειρά συχνότητας) είναι ίσο με μια σταθερά A. Βρείτε ποια τιμή της σταθεράς A (εξετάστε τις επιλογές: 0.1, 0.2, 0.3, …, 1.0) ταιριάζει καλύτερα με το συγκεκριμένο σύνολο κείμενων. Δημιουργήστε ένα διάγραμμα όπου ο άξονας x είναι η θέση ενός type σε φθίνουσα σειρά συχνότητας και ο άξονας y είναι η συχνότητα του type στην συλλογή. Και οι 2 άξονες να ακολουθούν την λογαριθμική κλίμακα. Δείξτε στο ίδιο διάγραμμα με διαφορετικά χρώματα (i) τις προβλέψεις του νόμου του Zipf (με την καλύτερη τιμή που βρέθηκε για τη σταθερά A) και (ii) τις πραγματικές μετρήσεις που έγιναν στα κείμενα.

Make sure you generate 2 separate functions: one to find the best A value and another to plot the diagrams. Make sure each function accepts and returns the necessary variables.“

The generated code is in my report notebook. No changes were necessary to be made.

B. N-gram Language Models

1

Language model	Original Text	Lowercase	Abstract digits
Bigrams (k = 1)	383.738	591.358	555.858
Bigrams (k = 0.01)	137.839	349.178	274.965
Trigrams (k = 1)	1352.323	1963.496	1740.481
Trigrams (k = 0.01	429.445	1024.961	796.219

Smoothing Parameter (k) In each row, $k = 0.01$ results in a significantly better (lower) perplexity. The $k = 1$ (Laplace) smoothing is too aggressive for this dataset. it reassigns too much probability mass from observed n-grams to unobserved ones, which increases the model's uncertainty (perplexity).

Bigrams vs. Trigrams Bigrams consistently outperform trigrams in every case. This is expected because our corpus is too small to create a large enough vocabulary of trigram combinations that we encounter in the test dataset. Hence, the models rely on the smoothing fallback during evaluation.

Lowecase Lowercasing increases perplexity. While it merges word variants (e.g., ”The” and ”the”), this causes more words to meet the frequency threshold of 3. This reduces the frequency of the “<UNK>“ token and makes the vocabulary larger. This forces the complicates the model which has to predict specific words instead of assigning “<UNK>“, thus increasing perplexity.

Abstract Digits This method shows an increased performance compared to the previous method. Digit abstraction leads to vocabulary shrinkage thus making the model less complex. All digit combinations collapse into a combination of “#“ tokens which the model does not have to learn, which leads to reduced complexity.

2

In general, all models produce mostly incoherent text due to the small vocabulary size. The Trigram models produced much more coherent sentences than the Bigrams.

Sentences generated with k=0.01 were far more realistic. At k=1, the probability distribution is too flat and the model frequently chooses random, unrelated words, resulting in less meaningful sentences.